Selecting Reliable LLM Reasoning

DASE7506 Mini-project 2 | Final experimental report | 31 October 2026

1. Research question and outcome

Can a compact candidate comparison network become more reliable by combining cross-candidate representations, training-only out-of-fold ranking and arithmetic evidence? The hypothesis is that representations capture broad plausibility, while explicit arithmetic can identify some locally inconsistent yet fluent solutions. Neither source should be treated as a complete proof of correctness.

Final official test accuracy: 385/467 = 82.4411%. This does not meet the requested target above 84.58%. The method needs at least 396 correct choices to exceed that target. It satisfies the measured local resource budgets: 9.901 MiB of inference assets and 6.010 seconds for fresh CPU prediction plus independent scoring. Reference-machine eligibility and audit-bank accuracy are not yet known.

Data and supervision

The unchanged supplied bank contains 4,755 training, 798 validation and 467 test questions, with four candidate solutions and 1,024-dimensional fixed embeddings per question. Candidate correctness is defined by the official final numeric-answer parser, not validity of every reasoning step. Multiple candidates may be correct. Every question has at least one correct choice. Data hashes and alignment passed the official checker.

All learned weights and standardisation statistics are fitted on training inputs/references only. Validation selects epochs, ensembles and correction strength. There are 2,114 training and 355 validation questions with at least two distinct valid parsed answers; all test questions have this property. Hard-validation accuracy is therefore the principal selection metric in later development.

Evaluation chronology and exposure

Frozen version	Correct / test	Accuracy
V1: two compact comparison nets	376 / 467	80.5139%
V2: six rich comparison nets	383 / 467	82.0128%
V3: rank + arithmetic corrections	385 / 467	82.4411%

The first public test score was known before V2 development, and both earlier aggregate scores were known before V3. No test reference solutions or per-example test errors were inspected for development. Subsequent model and setting selection used training/internal holdout/validation only; each reported version was frozen before scoring. These exposures are disclosed rather than presenting the final public test as a pristine unseen holdout. An independent audit remains necessary.

Comparison networks and answer evidence

2. Inputs and candidate scoring

The original comparison baseline concatenates the 1,024 supplied embedding dimensions with 24 features: parse validity, same-answer count, lengths, formatting and numeric counts, signed log answer magnitude, sign/zero/integrality, number of distinct answers and ten discourse-word counts. Rich features expand the non-embedding part to 206 dimensions using question quantities, question/solution word counts, punctuation, coarse equation checks and answer-relative numerical information.

A rich network reads [embedding, features], of dimension 1,230. A delta network reads [embedding, embedding minus group mean, features, features minus group mean], of dimension 2,460. Training means and standard deviations are used with floors of 0.1 and 0.03 respectively; normalised inputs are clipped to [-15,15]. The input contains no ID or candidate-position embedding.

A shared Linear(d,192)-LayerNorm-GELU-Dropout(0.3) encoder produces h. The head reads [h, h-m, m], where m is the mean of the four encodings, and applies Linear(576,96)-GELU-Dropout(0.25)-Linear(96,1). Shared weights and symmetric pooling permit arbitrary bank sizes and candidate permutations. A small shared linear residual, multiplied by 0.1, is added to each score.

Training objectives and selected members

Three objectives were compared: candidate binary cross entropy (BCE); pairwise softplus ranking plus 0.2 BCE; and correct-set probability mass plus 0.25 BCE. The mass loss is logsumexp(all logits) minus logsumexp(correct logits), so it rewards selecting any correct candidate rather than equal probabilities for every correct trace. This is aligned with the official scorer.

The final prior uses the mass objective, three rich members at seeds 300-302 and three delta members at seeds 400-402. Rich runs use AdamW learning rate 0.0004 and weight decay 0.025; delta runs use 0.0003 and 0.04. Batch size is 128. Training is 65 rich or 55 delta epochs, with validation selection. Hard questions receive weight 3; the delta runs use weight 0.25 for easier groups. The rich runs use weight 1 for easier groups. All runs are CPU only.

Family / seeds	Selected epochs*	Family weight
Rich mass: 300, 301, 302	19, 3, 6	0.25
Delta mass: 400, 401, 402	52, 19, 28	0.75

*Epoch numbers are zero-based. Member logits are averaged within each family and divided by the family mean within-question logit standard deviation on hard validation inputs. These input-derived scales are saved in config.json. Family weights are selected on hard validation. Invalid parsed answers are masked during answer comparison. Answer mixing averages half of a candidate score with half of its same-answer-group mean.

Out-of-fold ranking and arithmetic

3. Learning a correction without training-set leakage

A second learner must not train on unrealistically confident in-sample predictions from the first learner. We therefore use shuffled four-fold training splits (random state 1000). For each fold, one rich and one delta mass network train on the other three folds for 24 epochs. Their held-out scores form out-of-fold predictions for all training questions. Fold-only means and scales prevent feature-statistic leakage. Seeds are 1000 + fold + 10 x family index.

At meta-training, scores are centred within each question and divided by their standard deviation with a 0.05 floor. The meta-feature vector has 422 dimensions: one prior score, nine arithmetic-evidence features, the 206 rich features, and their 206 group-centred differences. CatBoost ranking is trained on hard training questions only, with four candidates grouped per question. No validation example fits a tree.

Two YetiRank models use seeds 1700 and 1701, depths 4 and 5, learning rate 0.025, L2 regularisation 20, 32 borders and four threads. Each run considers 1,600 trees. Validation selects 200 and 175 trees respectively. The mean rank output is multiplied by 2 and added to the normalised prior. The native NDCG@1 log uses the validation eval_set, not the public test bank.

Arithmetic evidence

The checker recognises conservative arithmetic expressions in math chunks or equation lines, including LaTeX fractions, multiplication/division, parentheses and small powers. It interprets a restricted Python syntax tree; it never executes candidate code. Unsupported variables and syntax are skipped. Equality uses a relative tolerance to accommodate small rounding differences.

When a numeric equation is inconsistent, a proposed corrected result can be propagated through later numeric expressions. A proposal earns support only when it matches another supplied candidate answer. Features include verified and failed equation counts, discrepancy sizes, support for a repaired answer and contradictions to the stated answer. This cannot detect all missed word-problem conditions. An intermediate mistake can coexist with a correct final answer, so arithmetic evidence is probabilistic.

Training repair proposals triggered on 163 hard questions and selected a correct answer on 79.75% of them. Direct final-equation proposals triggered on 121 and were 79.34% correct. This is insufficient for unconditional overrides. The final heuristic bonus is 0.5 x (repair support - 0.5 x contradiction count), added after rank-score scaling. The coefficient was selected on validation from {0,0.1,0.25,0.5,1}.

Frozen inference

Six neural members, two small tree models, two statistics files and one configuration are loaded from runs/final. A post-V3 numerical stability fix rounds meta features to four decimal places before tree prediction; this prevents a tiny upstream rounding difference from changing a tree branch in a permutation smoke check. The fixed precision was chosen for stability, not test accuracy. The 11-question permutation/new-ID check then passed. Validation improved by one question; test remained 385/467. The earlier V3 aggregate score was known before this fix. This additional test exposure is disclosed. The selector reads only the supplied bank and embeddings, computes evidence, then returns one ordinary Python integer per question. It performs no LLM generation, network request, lookup by ID or retraining. All required learned statistics and weights are included in the inference bundle.

Experiments and reliability

4. Controlled comparisons and limitations of ablations

Validation method	All / 798	Hard / 355
Parsed-answer majority vote	86.22%	69.01%
Original 24-feature logistic, C=1	90.10%	77.75%
Embedding logistic, C=0.1	89.35%	76.62%
V1 two-net prior, answer mixing	93.98%	86.48%
V2 six-net mass prior	94.49%	87.61%
V3 prior + mean ranking correction	94.86%	88.45%
V3 plus arithmetic bonus, unrounded	95.24%	89.30%
V3.1 fixed numerical precision (final)	95.36%	89.58%

The two V3 rows before the precision fix differ by the arithmetic bonus alone: it improves hard validation by three questions (0.85 percentage points). Adding ranking to the V2 prior also improves hard validation by three questions. These small effects warrant caution; the much larger improvement over majority voting does not establish complete mathematical verification.

The original baseline search compares C in {0.01,0.1,1,10}. Additional families include rich and delta BCE/pair/mass networks, two raw-feature boosted classifiers, two arithmetic-program boosted classifiers, hashed word/character text models, PCA RBF support-vector models, contrastive PCA polynomial/RBF kernels, hard-only networks, answer-group networks and attention networks. They are documented in per-family records. Many were worse; larger or more elaborate models did not reliably help.

For example, the best raw-feature tree model reached 83.38% hard validation, the hashed text model 65.07%, the answer-group single models at most 85.63%, and the attention single models at most 85.35%. The best original PCA SVM reached 82.82%. These are family-level results under their own searches, not matched architectural ablations. No causal claim about attention or text being inherently unhelpful follows.

Internal holdout and uncertainty

A separate hard-only experiment reserves 20% of hard training questions, using seed 700. Two internal runs select a fixed 34-epoch budget from their mean internal-holdout accuracy, then four models fit all hard training questions. Their hard-validation accuracies range from 83.94% to 86.48%, below the final prior. The experiment demonstrates an internal development protocol but does not independently validate every subsequently selected component.

Candidate-margin confidence is unreliable: multiple candidates can share a correct answer, making top-two margins tiny on easy questions, while invalid-answer masking can produce huge margins. Earlier ascending margin quartiles had accuracies 98.50%, 98.50%, 94.47% and 84.42%; the largest mean margin was sentinel-dominated. These margins must not be interpreted as calibrated probabilities. A separate calibration set and answer-group confidence analysis remain future work.

Generalisation, failures and resources

5. What did not transfer

Final validation makes 37 errors, all in the hard subset; test makes 82 errors. The gap between hard-validation accuracy (89.58%) and test accuracy (82.44%) is 7.14 points. This is consistent with validation-selection optimism and difficulty differences, but the current experiments cannot separate their contributions. Extensive validation search is a material limitation. The exposed public test is not a substitute for an independent audit bank.

Validation error inspection motivated generic arithmetic evidence, rather than ID-specific rules. Examples of error types are an incorrect sum despite correct listed terms, an incorrect fraction conversion, forgetting an extra coat of paint, confusing revenue with profit, and returning an intermediate subtotal instead of the requested total. The arithmetic checker addresses some calculation errors; missed conditions often remain internally consistent and require semantic interpretation.

No per-example test errors were inspected to construct corrective rules. The final score gain from V2 to V3 is two questions (0.43 points), smaller than the validation gain. It would be misleading to claim that the larger development improvement has been confirmed on test. The requested ranking/accuracy outcome is not achieved.

Resource accounting

Measured quantity	Value	Eligibility
Final uncompressed inference assets	10,382,427 bytes	9.901 MiB < 16
Fresh prediction + scoring wall time	6.010 seconds	Local < 30 s
Official runner prediction time	5.087 seconds	Excludes startup
Official scorer time	0.043 seconds	Separate process

Hardware: Intel Core i7-12700H, Windows 11, CPU only. Versions: Python 3.13.5, NumPy 2.2.6, PyTorch 2.7.1+cpu, scikit-learn 1.6.1, CatBoost 1.2.10. OMP/OpenBLAS/MKL/NumExpr limits and Torch inference threads are four. Full fresh-process timing includes imports, loading and preprocessing. Instructor eligibility is measured on Xeon Platinum 8457C; that machine was not available locally.

Training sometimes ran several four-thread CPU jobs concurrently; inference measurement ran after training jobs completed. Per-family elapsed training/search times, seeds, selected epochs/trees and settings are in runs/training.json and runs/v2/*records.json. These elapsed job times overlap and should not be summed as total wall time or interpreted as CPU core-seconds. No GPU or paid API was used. The full development workflow may take tens of minutes or longer on another CPU.

The inference checkpoint link should target runs/final only. Unselected experimental checkpoints are omitted from the final package and can be regenerated. Historical validation predictions and experiment records are training/development evidence, not inference inputs. All required inference files are enumerated with SHA-256 hashes in frozen_final.json.

Reproduction and disclosure

6. Reproduce the frozen score

Follow the repository README installation commands, then run from code/:

```
python predict.py --method submission_final.py --checkpoint runs/final --split test --output runs/reproduced-test-predictions.json

python score_predictions.py --split test --predictions runs/reproduced-test-predictions.json --output runs/reproduced-test-score.json
```

Expected official accuracy: 82.44111349036403. Retraining is unnecessary for reproduction. The exact checkpoint/configuration and inference-source hashes are recorded in runs/frozen_final.json. The retained official parser, runner, scorer and data checker have been compared byte-for-byte with the supplied originals. The official hash checker verifies the original datasets.

Reproduce development

Run train_all.ps1 from code/ to replay the recorded training, internal holdout, out-of-fold fitting and validation selection. Individual Python scripts correspond to each experiment family. Source code and records specify seeds, optimisation, epoch budgets, feature definitions and searches. Training artifacts are written separately from runs/final. No training script loads test references. Cross-platform retraining can yield numerical differences; the supplied weights are the claimed-score reproduction target.

The course recommends Python 3.12; this local experiment used 3.13.5. Cross-version reproduction has not been tested. The final inference function accepts arbitrary input bank sizes and the official audit-bank options. No hard-coded sample counts, question IDs or cached predictions participate in inference. Saved test predictions are evidence for the independent scorer only.

AI and reuse disclosure

OpenAI Codex substantially assisted with implementation, feature/model design, experiment execution, analysis, documentation and report preparation. The student must review and understand the work before submitting it. The official course data and evaluation tooling were reused unchanged. Questions and references derive from GSM8K under its retained MIT license; candidate traces and fixed representations were supplied by the course using Qwen3-0.6B. No additional pretrained weights or datasets were used.

Sources and submission status

Course guide: https://github.com/xudongwu-0/DASE7506-MP2/blob/main/guide/GUIDE.md
Official running instructions: https://github.com/xudongwu-0/DASE7506-MP2/blob/main/code/README.md
Numerical evidence: bundled official score JSON files, resource records and development records.

The code and frozen checkpoint are prepared for student review and publication. No public repository, checkpoint URL or GitHub submission issue has been created because student ID and a public GitHub destination have not been provided. Publish the exact code version and its matching runs/final checkpoint, then create the issue prepared by the MP2 website before 31 October 2026, 23:59 UTC+8. Self-report 82.44111349036403, never a validation score. No high ranking is claimed.